

Calibrated Trust in Agentic Coding

A working framework for staying a competent engineer while using AI that can outrun your understanding.

The real shift

Agentic coding doesn't eliminate the gap between what you wrote and what you understand — it just makes you hit that gap on day one instead of year three. No engineer on a large team understands 100% of the codebase either; they understand interfaces and invariants and trust the tests. The skill to build isn't "understand everything." It's knowing how to quickly build justified confidence in code you didn't write.

Four habits that convert faith into verification

Interrogate, don't just accept. Ask the AI to explain its own code back to you design-review style: what could break this, what did it consider and reject, what's the failure mode if an assumption is wrong. A model that understands its own code answers this consistently. A vague or shifting answer is a signal, even if you can't spot the bug yourself.

Demand minimal repros for anything confusing. If a bug or fix doesn't make sense, don't accept the patch — ask for the smallest possible example that reproduces the failure. Tracing 10 lines by hand teaches you more than reading a 200-line diff, and it's fast.

Let tests carry the understanding you can't hold in your head. Write or review tests for the exact edge cases you're fuzzy on. This turns "I trust this because Claude says so" into "I trust this because I watched it fail before the fix and pass after."

Calibrate trust by domain, not globally. Lean on AI heavily where correctness is cheap to verify externally (output matches, state updates correctly, tests pass). Stay hands-on where bugs are silent and expensive — concurrency, resource allocation, security, anything touching money or persistent state.

A quick self-check before accepting a diff

Question	If no — do this
Can I explain why this fix works, not just that it works?	Ask for the design-review explanation above
Does a test exist that would fail if this were wrong?	Write or request one for the specific edge case
Is this in a high-stakes area (state, concurrency, security)?	Trace it by hand or build a minimal repro
Would I be comfortable explaining this in a code review?	Keep asking questions until you would be

The bottom line: dependence isn't the risk — unverified dependence is. Every time you push a diff through one of these checks instead of taking it at face value, you're doing the thing senior engineers already do on codebases too large to fully hold in their heads.